

Tiling Game

Barchin e Charos estão a jogar um jogo numa grelha de células quadradas unitárias de tamanho $2N \times 2M$. As linhas são numeradas de 0 a $2N - 1$ de cima para baixo, e as colunas são numeradas de 0 a $2M - 1$ da esquerda para a direita. Para $0 \leq i < 2N$ e $0 \leq j < 2M$, denotamos a célula da linha i e da coluna j por (i, j) .

Barchin fornece a Charos $N \cdot M$ **blocos**, um a um. Cada bloco é um quadrado de 2×2 composto por quatro **peças** de 1×1 . Barchin coloriu cada peça do bloco de preto ou branco, garantindo que **pelo menos uma peça é branca**.

Charos deve colocar cada bloco na grelha **imediatamente após o receber**, sem saber que blocos irá receber posteriormente. Os blocos não podem ser rodados. Cada bloco deve ser colocado inteiramente dentro da grelha, cobrindo exatamente quatro células da grelha. Além disso, a peça superior esquerda de cada bloco deve cobrir uma célula cujas coordenadas de linha e coluna sejam ambas pares. Cada célula da grelha pode ser coberta por, no máximo, um bloco.

Barchin ganha o jogo se, após a colocação de qualquer bloco, existir um quadrado de células de 2×2 coberto por quatro peças pretas. Formalmente, se as células (a, b) , $(a + 1, b)$, $(a, b + 1)$ e $(a + 1, b + 1)$ estiverem todas cobertas por peças pretas para algum $0 \leq a < 2N - 1$ e $0 \leq b < 2M - 1$, então Barchin ganha. Os índices a e b não têm de ser pares.

Charos ganha se colocar todos os blocos $N \cdot M$ sem que Barchin ganhe uma única vez. Nota que a colocação de $N \cdot M$ blocos cobrirá completamente a grelha.

A tua tarefa é implementar uma estratégia para que Charos ganhe o jogo. Pode-se provar que, sob as restrições dadas, Charos pode sempre posicionar os blocos de forma a garantir a vitória, independentemente das cores dos blocos que receber posteriormente.

Detalhes da Implementação

Deves implementar as seguintes duas funções:

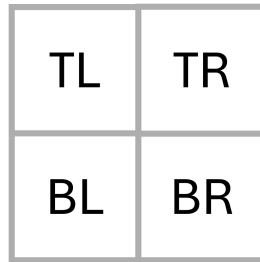
```
void init(int N, int M)
```

- N : metade do número de linhas da grelha.
- M : metade do número de colunas da grelha.

- A função é chamada exactamente uma vez por caso de teste, no início da execução do teu programa.

```
std::pair<int, int> receive_block(int TL, int TR, int BL, int BR)
```

- TL , TR , BL , BR : a cores das células dos quatro cantos (as peças nas posições *top-left*, *top-right*, *bottom-left*, e *bottom-right* do bloco atual, respetivamente, como mostrado na figura seguinte. Cada valor é 0 (branco) ou 1 (preto).
- Esta função é chamada exactamente $N \cdot M$ vezes por caso de teste, após a chamada inicial a `init`.



Esta função deve devolver um par de inteiros (i, j) , onde i é a coordenada da linha e j é a coordenada da coluna da célula onde deve ser colocada a peça superior esquerda deste bloco. Tanto i como j devem ser pares, e a região 2×2 coberta pelo bloco não deve sobrepor-se a nenhum bloco colocado anteriormente.

Se `receive_block` alguma vez retornar um par que não satisfaça estes requisitos, ou se depois de colocar o bloco um quadrado de células de 2×2 ficar completamente coberto por peças pretas, o avaliador terminará imediatamente o teu programa e o veredicto para o caso de teste será `Output isn't correct`.

O comportamento do avaliador **não é adaptativo**. Isto significa que a sequência de blocos que Barchin fornece a Charos é fixada antes da chamada a `init`.

Restrições

Para cada bloco, seja S o número de peças pretas entre as suas quatro peças. Ou seja, $S = TL + TR + BL + BR$.

- $1 \leq N, M \leq 100$
- $0 \leq S \leq 3$ para cada bloco.

Subtarefas

Subtarefa	Pontos	Restrições Adicionais
1	6	$S = 1$ para cada bloco e $N = 2$.
2	16	$S = 3$ para cada bloco. $N = M$, N é par e cada uma das quatro colorações de blocos possíveis aparece exactamente $\frac{N^2}{4}$ vezes.
3	10	$S = 1$ para todos os blocos.
4	29	$S \leq 2$ para todos os blocos.
5	39	Sem restrições adicionais.

Exemplo

Considera um jogo com $N = 1$ e $M = 2$, ou seja, a grelha tem 2 linhas e 4 colunas. O avaliador chama primeiro:

```
init(1, 2)
```

Inicialmente, todas as células estão vazias. A grelha tem a seguinte aparência:

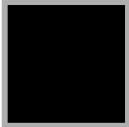
	0	1	2	3
0				
1				

Existem $N \cdot M = 2$ blocos para colocar. Supõe que Barchin fornece um bloco com três peças pretas e uma peça branca no canto superior direito. O avaliador chama:

```
receive_block(1, 0, 1, 1)
```

Charos decide colocar este bloco no lado esquerdo da grelha devolvendo $(0, 0)$.

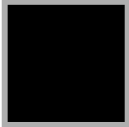
A grelha está agora assim:

	0	1	2	3
0				
1				

De seguida, Barchin apresenta outro bloco com três peças pretas e uma peça branca no canto superior esquerdo:

```
receive_block(0, 1, 1, 1)
```

A única célula restante com um número par de linhas e colunas que pode servir como canto superior esquerdo de um bloco de 2×2 é $(0,2)$, pelo que Charos devolve $(0,2)$. A grelha fica assim:

	0	1	2	3
0				
1				

Nenhum quadrado 2×2 está completamente coberto por peças pretas, pelo que Charos conseguiu posicionar todos os blocos sem que Barchin vencesse. Charos ganha o jogo.

Avaliador de Exemplo

Formato de Input:

```
N M
TL[0] TR[0] BL[0] BR[0]
TL[1] TR[1] BL[1] BR[1]
...
TL[NM-1] TR[NM-1] BL[NM-1] BR[NM-1]
```

Formato de Output:

```
R[0] C[0]  
R[1] C[1]  
...  
R[NM-1] C[NM-1]
```

Aqui, $R[k]$ e $C[k]$ são o par de inteiros devolvido pela k -ésima chamada a `receive_block`.